

Intermediate (make search conditional) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper Python), **Advanced** (how a real router handles the messy edge cases). Read Basic first even if you already know Python — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the exact pieces

Basic Version

Right now, your graph either always searches or leaves it up to the model to decide inside one call. This exercise wants a separate decision step, made by the graph itself, before the model ever gets to reason: "does this specific task need the document search, or not?"

Think of it as a fork in the road that happens right after the task comes in, not something buried inside the model's own reasoning.

Things to use:

- A plain function that looks at the task and returns a short label, like `"search"` or `"skip"`.
- `graph.add_conditional_edges(from_node, routing_function, {"search": "search_node_name", "skip": "reason_node_name"})`.
- Two test questions: one that clearly needs your documents, one that clearly doesn't (like "hello, how are you?").

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

This is Doc09's `add_conditional_edges` mechanism, applied to a real decision: "route to search, or skip it."

A conditional edge needs a plain Python function that looks at the current state and returns the *name* of the next node to go to — it doesn't do any work itself, it just decides:

```
def should_search(state: dict) -> str:
    ...
    return "search" # or "skip"
```

Then you register it:

```
graph.add_conditional_edges("router", should_search, {"search": "search_node", "s
```

The exact pieces:

- **The routing function's return value** — it must exactly match one of the keys in the mapping dict you pass to `add_conditional_edges`. A typo here fails silently in confusing ways (LangGraph raises an error about an unknown node, which can look unrelated to the real cause).
- **Where the decision lives** — a plain keyword/heuristic check is a fine first version; a second, smaller model call classifying "needs search: yes/no" is the more robust real-world version, at the cost of one extra call.
- **The two mapped branches** — one path goes to your search node, the other skips straight to your reasoning node, both eventually rejoining before the final answer.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

A keyword list and a single model call both share the same weakness: they're each wrong in a different, predictable direction. A keyword list misses real search-needing questions phrased without its exact words ("what's our time-off rule?" instead of "policy"). A model call catches those, but costs one extra round trip on *every single task*, even the 90% that are obviously one way or the other — and it isn't perfectly deterministic either, so the same question can route differently on different runs.

The real design question isn't "keywords or a model call" — it's "what's the cheapest check that's right most of the time, and when is it worth paying for a more expensive, more accurate check instead?"

A hybrid answers that directly: run the cheap keyword check first. If it's confident (a clear keyword hit, or a clearly conversational greeting with no keywords and no question mark), trust it and skip the model call entirely. Only fall back to the model classification call for the genuinely ambiguous middle — task text with no keyword hit that still isn't obviously a greeting.

The extra pieces needed for a hybrid router:

- A "confident skip" check — something unmistakably conversational, like a short message with no question mark and no keyword hit, routes straight to `"skip"` without ever calling the model.
- A "confident search" check — a clear keyword hit routes straight to `"search"`, same reasoning.
- Only the leftover, ambiguous cases fall through to the one-line model classification call from Hint 2's Advanced Version.
- A cost/accuracy note worth writing down: this hybrid still isn't perfect (there's no free lunch here) — it just spends the expensive check only on the tasks where the cheap one is least trustworthy.

Sketch which cases you'd route with keywords alone, and which you'd fall through to the model, before Hint 2.

Difference between Basic, Intermediate, and Advanced: Basic names the conditional-edge mechanism and the two branch labels. Intermediate shows the real `should_search` signature and wiring, and names the keyword-vs-model tradeoff as a single either/or choice. Advanced treats it as a spectrum instead of a binary choice — cheap and fast for the clear cases, falling through to the more expensive, more accurate check only for the genuinely ambiguous ones — which is the difference between a router that's "good enough for most" and one that's deliberately built to spend its accuracy budget where it actually matters.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
function should_search(state):
    look at the task text
    if it contains words like "document", "policy", "according to" (or similar cl
        return "search"
    otherwise:
        return "skip"

wire it up:
    add_conditional_edges(from the router node, should_search, {"search": search_

test:
    run with a question mentioning your documents -> should route to search
    run with "hello, how are you?" -> should skip straight to reasoning
```

Here's almost the whole thing — just try running it and reading it line by line:

```
def should_search(state):
    task = state["messages"][-1].content.lower()
    if "document" in task or "policy" in task:
        return "search"
    return "skip"

graph.add_conditional_edges("router", should_search, {"search": "search_node", "s
```

Expected output if you run just this: nothing on its own — invoke the graph with a document-mentioning question and a plain greeting, and check which node each one lands on.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
def define:
  def should_search(state: dict) -> str:
    task = state["messages"][-1].content.lower()
    keywords = ["document", "policy", "according to", "the file says"]
    if any(keyword in task for keyword in keywords):
      return "search"
    return "skip"

  register:
    graph.add_conditional_edges(
      "router",
      should_search,
      {"search": "search_node", "skip": "reason_node"},
    )

  test:
    invoke with a document-referencing question -> confirm it went through search
    invoke with an unrelated greeting -> confirm it skipped straight to reason_no
```

```
def should_search(state: dict) -> str:
  task = state["messages"][-1].content.lower()
  keywords = ["document", "policy", "according to"]
  if any(keyword in task for keyword in keywords):
    return "search"
  return "skip"
```

A keyword check is a reasonable first version but is easy to fool. Write the

`add_conditional_edges` wiring and both test invocations yourself, then compare against the [Solution](#).

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

```
function should_search(state):
    task = the task text
    if task has a keyword AND is short (no question mark): return "search"    # co
    if task has no keyword AND looks conversational (short, no "?"): return "skip
    otherwise (ambiguous):
        ask a small model call: "does this need document search? yes/no"
        return "search" if yes else "skip"
```

Here's almost the whole thing — fill in the missing piece yourself:

```
from langchain_core.messages import HumanMessage

KEYWORDS = ["document", "policy", "according to", "the file says"]

def should_search(state: dict) -> str:
    task = state["messages"][-1].content
    lowered = task.lower()
    has_keyword = any(keyword in lowered for keyword in KEYWORDS)

    if has_keyword:
        return "search"

    looks_conversational = len(task.split()) <= 6 and "?" not in task
    if looks_conversational:
        return "skip"

    # your turn: the ambiguous middle case -- fall through to a one-line
    # model classification call, the same pattern as Hint 1's Advanced note,
    # and return "search" or "skip" based on its answer
    ...
```

Fill in the fallback classification call yourself, then compare all 3 of your finished versions against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: Basic's pseudocode and near-complete code always return based on one keyword check, no matter how ambiguous the task. Intermediate is the same shape, described more precisely, with a broader keyword list — still just one check, no

fallback. Advanced adds two confident short-circuits (a clear keyword hit, a clearly conversational message) and only falls through to a model call for whatever's left over — the same idea Hint 1's Advanced Version describes, made concrete: spend the expensive check only where the cheap one is genuinely unsure.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)